Python 101 — Quick Reference Guide

Click + on any entry to expand options, descriptions, and examples | All 4 weeks / 38 lessons | Generated 2026-06-09

Output & Input

- print(value) # outputs value(s) to the screen
  - print("hello", "world") — multiple args separated by space by default
  - print("a", "b", sep="-") — custom separator → a-b
  - print("loading", end="...") — custom ending (default is newline \n)
  - print("ha" * 3) — string repetition → hahaha
  - print("Hi " + name) — concatenate with +

```
print("Name:", name, "Score:", score)
print("done", end="")  # no newline at end
```

- input("prompt") # reads a line from the user — ALWAYS returns a string
  - name = input("Enter your name: ") — stores result in variable
  - age = int(input("Age: ")) — convert to int immediately
  - price = float(input("Price: ")) — convert to float

⚠ input() ALWAYS returns a string — wrap in int() or float() for numbers

```
name = input("Your name: ")
age  = int(input("Your age: "))
```

Variables & Data Types

- variable = value # stores a value; use snake_case names
  - Names: letters, digits, underscores — cannot start with a digit
  - Convention: my_score, player_name, is_active
  - = is assignment; == is comparison — never mix them
  - Re-assigning overwrites the old value

```
score  = 0        # int
price  = 9.99     # float
name   = "Ron"    # str
active = True     # bool
```

- str int float bool # the four basic data types
  - str — text in quotes: "hello", 'world'
  - int — whole number: 42, -7, 0
  - float — decimal number: 3.14, -0.5
  - bool — True or False (capitalized; storable like any variable)
- type(value) # returns the data type of a value — useful for debugging
  - type(42) → <class 'int'>
  - type("hi") → <class 'str'>
  - type(3.14) → <class 'float'>
  - type(True) → <class 'bool'>
- int(x) float(x) str(x) # type conversion functions
  - int("42") → 42 — string to integer
  - float("3.14") → 3.14 — string to float
  - str(100) → "100" — number to string (required before string concatenation)
  - int(9.9) → 9 — truncates, does NOT round

  ⚠ int("hello") raises ValueError — only works on valid numeric strings

```
age = int(input("Age: "))
msg = "Score: " + str(score)  # must convert to concat
```

➕ Arithmetic & Math

- + - * / // % ** # arithmetic operators
  - + addition: 5 + 3 → 8
  - - subtraction: 10 - 4 → 6
  - * multiplication: 3 * 4 → 12
  - / division (always float): 7 / 2 → 3.5
  - // floor division (drops decimal): 7 // 2 → 3
  - % modulo (remainder): 7 % 3 → 1
  - ** exponent (power): 2 ** 8 → 256

  ⚠ number % 2 == 0 checks EVEN; number % 2 alone is truthy for ODD numbers

```
result = 10 / 3    # 3.333...  (float)
result = 10 // 3   # 3         (floor)
result = 10 % 3    # 1         (remainder)
result = 2 ** 10   # 1024      (power)
```

- round(number, digits) # rounds number to N decimal places

- round(3.14159, 2) → 3.14
- round(3.7) → 4 — rounds to nearest integer when no digits given
- round(2.5) → 2 — Python uses banker's rounding (rounds to even)

```
price = round(total / qty, 2)  # 2 decimal places
```

+ f"{variable}" # f-string: embed variables/expressions directly in a string

⚠️ Common Errors

- TypeError # wrong type used in an operation
  - Caused by: "score: " + 95 — can't concatenate str + int
  - Fix: "score: " + str(95) or use an f-string
- NameError # using a variable before it's defined
  - Caused by: print(score) before score = 0 is set
  - Fix: define the variable before using it
- ZeroDivisionError # dividing by zero
  - Caused by: 10 / 0 or 10 // 0
  - Fix: check if divisor != 0: before dividing
- ValueError # conversion fails because the value is wrong type or format
  - Caused by: int("hello") — can't convert non-numeric string
  - Also: int("3.14") — must go through float first: int(float("3.14"))
- IndentationError # code block not indented correctly
  - Python uses 4 spaces per indent level (tabs or spaces — never mixed)
  - All code inside a function, if block, or loop must be indented one level
- UnboundLocalError # variable used before assigned inside a function
  - Caused by reading a global variable then trying to assign to it locally
  - Fix: pass values as parameters instead of relying on globals
- IndexError: list index out of range # index position doesn't exist
  - Caused by: items[5] on a 5-item list (valid range: 0–4)
  - Valid indexes: 0 to len(list) - 1
  - Fix: guard with if index < len(list): before accessing

🔧 Functions

- def function_name(param): # define a reusable named block of code

- Body must be indented (4 spaces)
- Parameters = variable names in the () at definition
- Arguments = actual values passed when calling the function
- Multiple parameters: def add(a, b):
- No params: def greet():
- Functions implicitly return None if no return statement

```
def greet(name):
    print(f"Hello, {name}!")

greet("Ron")  # argument is "Ron"
```

- return value # exits the function and sends a value back to the caller
  - return bare or return None — exits without a value
  - Code after a return never runs
  - Multiple return paths are fine — only one runs per call
  - Capture returned value: result = my_function(x)

```
def add(a, b):
    return a + b

total = add(3, 4)  # total = 7
```

- guard clause # early return at the top of a function to handle edge cases first
  - Check for invalid/special cases at the top, return early
  - Avoids deeply nested if/else — keeps the main logic flat and readable

```
def divide(a, b):
    if b == 0:
        return None   # guard: handle bad input first
    return a / b      # main logic
```

- local scope # variables inside a function only exist inside that function
  - Variables defined inside a function disappear when the function ends
  - Can READ a global variable from inside a function
  - Cannot ASSIGN to a global variable without global keyword — creates a new local one instead
  - Assigning then reading causes UnboundLocalError
  - Best practice: pass values in as parameters instead of relying on globals

🔀 Control Flow — if / elif / else

- if condition: elif condition: else: # branching — only ONE branch runs
  - if — first check; runs if condition is True
  - elif — checked only if all prior conditions were False
  - else — runs when no condition matched (no condition needed; optional)
  - "First true wins" — order matters; put specific conditions before general ones

```
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
elif score >= 60:
    grade = "D"
else:
    grade = "F"
```

- < > <= >= == != # comparison operators — return True or False
  - < less than > greater than
  - <= less than or equal >= greater than or equal
  - == equal to (comparison) — NOT = which is assignment
  - != not equal to
  - Chained comparison: 10 <= temp < 25 — both conditions in one expression
- and or not # logical operators to combine conditions
  - and — True only if BOTH sides are True
  - or — True if EITHER side is True
  - not — flips True ↔ False
  - Short-circuiting: and stops at first False; or stops at first True

```
if age >= 18 and has_id:
    print("allowed")
if not is_banned:
    print("welcome")
```

- is / is not # identity check — preferred way to test for None
  - x is None — preferred over x == None
  - x is not None — confirms something has a value
  - == checks value equality; is checks same object in memory

```
if result is None:
    print("no result")
```

- "text" in string # checks if a substring (or value) exists inside a string or list
  - "cat" in "concatenate" → True
  - "cat" not in "dog" → True
  - Case-sensitive: "Cat" in "cat" → False
  - Works on lists too: 5 in [1, 3, 5] → True
- decision table → if/elif/else # turn rows of a logic table into branches
  - Each table row becomes one if / elif branch
  - Check for overlaps: one value shouldn't match two rows
  - Check for gaps: every possible value should match at least one row
  - Flat if/elif/else is better than deeply nested ifs

🔁 Loops

- for i in range(stop) # loop from 0 to stop-1
  - range(5) → 0, 1, 2, 3, 4
  - range(start, stop) → start to stop-1
  - range(start, stop, step) → with custom step
    - range(1, 11) → 1 through 10 (stop 11 not included)
    - range(0, 10, 2) → 0, 2, 4, 6, 8 (evens)
    - range(10, 0, -1) → 10, 9, 8, … 1 (countdown)

```
for i in range(5):
    print(i)          # 0 1 2 3 4

for i in range(1, 6):
    print(i)          # 1 2 3 4 5
```

- for item in list: # loop directly over list values — no index needed
  - Use when you only need the value, not the position
  - Cleaner and shorter than range(len(list)) when index isn't used
  - for i in range(len(list)): — use this version when you need the index too

```
# value-based (preferred when no index needed)
for score in scores:
    print(score)

# index-based (when position matters)
```

```
for i in range(len(scores)):
    print(i, scores[i])
```

- accumulator pattern # initialize a running total to 0 before loop, add inside
  - Always initialize accumulator BEFORE the loop
  - total = total + value can also be written total += value
  - Works for sums, products, counts, building strings, building lists

```
total = 0
for i in range(1, 11):
    total = total + i
print(total)  # 55

# average from a list
total = 0
for num in numbers:
    total += num
average = total / len(numbers)
```

- counter pattern # count how many items meet a condition inside a loop
  - Initialize counter = 0 before loop
  - Inside loop: if condition: counter += 1
  - Can combine counter + total accumulators in one loop

```
even_count = 0
even_total = 0
for num in numbers:
    if num % 2 == 0:   # % 2 == 0 means EVEN
        even_count += 1
        even_total += num
```

- while condition: # repeats while condition is True; use when repeat count is unknown
  - Pattern: initialize → test → update
  - Must update the condition variable inside the loop or it runs forever
  - while True: — intentional infinite loop; must use break to exit

```
# password checker
password = input("Password: ")
while password != "opensesame":
    print("Incorrect")
    password = input("Try again: ")
print("Access granted")
```

- break # exits the loop entirely — no more iterations
  - Works in both for and while loops
  - Only exits the innermost loop it's in
  - Common pattern: while True: ... if done: break

```
score = 0
while score != -1:
    score = int(input("Score (-1 to quit): "))
    if score == -1:
        break
    print("Recorded")
```

- continue # skips the rest of the current iteration and starts the next one
  - Loop does NOT exit — only this iteration is skipped
  - Useful to filter out items that don't meet a condition

⚠ In a while loop, make sure the update variable changes before continue or you get an infinite loop

```
for num in range(1, 11):
    if num % 2 != 0:
        continue       # skip odds
    print(num)         # only prints evens
```

🔤 Strings

- string[index] # get a single character by position — 0-based
  - First character is index 0, not 1
  - word[-1] — last character (negative counts from end)
  - word[-2] — second-to-last

```
word = "Python"
# + indexes:  0  1  2  3  4  5
# - indexes: -6 -5 -4 -3 -2 -1
word[0]   # "P"
word[2]   # "t"
word[-1]  # "n"  (last)
word[-2]  # "o"  (second-to-last)
```

- string[start:stop] # slice: get a substring — start included, stop excluded
  - word[1:4] on "Python" → "yth" (indexes 1, 2, 3)
  - word[:3] — from start up to index 3 (not included)
  - word[3:] — from index 3 to the end

- word[:] — copy of the whole string
- word[start:stop:step] — every Nth character
  - word[0:6:2] on "Python" → "Pto" (every 2nd char)
  - word[::-1] → reversed: "nohtyP"
- Negative slices: word[-3:] → last 3 chars; word[:-2] → all but last 2

```
word = "Python"
word[1:4]    # "yth"
word[:3]     # "Pyt"
word[3:]     # "hon"
word[::-1]   # "nohtyP"  (reversed)
word[-3:]    # "hon"  (last 3 chars)
```

- string.strip() # removes leading and trailing whitespace; returns new string
  - " hello ".strip() → "hello"
  - Does NOT modify the original — assign the result
  - After strip(), empty result "" signals "no data / missing field"

```
name  = " Ron Hensley "
clean = name.strip()    # "Ron Hensley"

field = parts[1].strip()
if field == "":
    print("no data")
```

- string.lower() / string.upper() # returns new all-lowercase / all-uppercase string
  - "Hello".lower() → "hello"
  - "hello".upper() → "HELLO"
  - Use .lower() before comparisons for case-insensitive matching

```
answer = input("Yes or no? ").lower()
if answer == "yes":
    ...
```

- string.replace(old, new) # returns new string with ALL occurrences of old swapped for new
  - Case-sensitive: "Hello".replace("h", "J") → "Hello" (no change)
  - Replaces ALL occurrences, not just the first
  - Returns a new string — original is unchanged

```
msg   = "server down"
fixed = msg.replace("server", "database")
# "database down"
```

- string.find(sub) # returns index of first match, or -1 if not found (case-sensitive)
  - "hello".find("ll") → 2
  - "hello".find("z") → -1 (not found)
  - Case-sensitive: "URGENT".find("urgent") → -1
  - Use if text.find("x") != -1: or simpler: if "x" in text:
- string.split(sep) # splits string at separator, returns a list of pieces
  - "a,b,c".split(",") → ["a", "b", "c"]
  - "hello world".split(" ") → ["hello", "world"]
  - Result is a list — access pieces by index

```
record = "Ron,Hensley,ron@example.com"
parts = record.split(",")
first = parts[0]  # "Ron"
email = parts[2]  # "ron@example.com"
```

- "sep".join(list) # glues a list of strings together with separator between each item
  - ", ".join(["a","b","c"]) → "a, b, c"
  - " | ".join(parts) → pipe-separated string
  - "".join(chars) → join with no separator

```
parts  = ["Ron", "Hensley", "ron@x.com"]
result = " | ".join(parts)
# "Ron | Hensley | ron@x.com"
```

- method chaining # call multiple string methods in a row — left to right
  - Each method returns a new string which the next method acts on
  - Order matters — strip first, then lower, then replace

```
text  = " URGENT: reset password "
clean = text.strip().lower().replace(": ", ":")
# "urgent:reset password"
```

📋 Lists

- my_list = [a, b, c] # create a list — ordered, mutable, any mix of types
  - Square brackets, comma-separated values

- Can hold ints, floats, strings, bools — even mixed types
- One variable holds many related values — better than score1, score2, score3…
- list(range(start, stop)) — create list of integers from a range

```
scores = [10, 15, 8, 20]
names  = ["Alice", "Bob", "Carol"]
nums   = list(range(1, 6))  # [1, 2, 3, 4, 5]
```

- len(list) # returns the number of items in the list (also works on strings)
  - len([5, 6, 7, 8]) → 4
  - len("hello") → 5
  - Used in range(len(list)) for index-based loops
  - Valid indexes: 0 to len(list) - 1
- list[index] # access a single item by position — same 0-based rules as strings
  - scores[0] — first item
  - scores[-1] — last item
  - scores[2] — third item (index 2)
  - list[start:stop] — slice returns a new sub-list (same rules as strings)
    - items[:3] → first 3 items
    - items[2:] → from index 2 to end
    - items[:] → shallow copy of entire list

```
scores = [80, 95, 72, 88]
scores[0]    # 80
scores[-1]   # 88 (last)
scores[1:3]  # [95, 72]
```

- list[index] = value # replace an item in place — lists are mutable, strings are not
  - Changes the original list — no new list is created
  - "Mutable" means changeable in place
  - Strings are immutable — word[0] = "X" raises a TypeError

```
colors = ["red", "green", "blue"]
colors[1] = "yellow"
# ["red", "yellow", "blue"]
```

- list.append(x) # adds x to the END of the list; modifies in place, returns None
  - Adds exactly one item to the end
  - Does NOT return a new list — it changes the original
  - Most common list method; great for building lists one item at a time

```
trades = []
trades.append("AAPL")
trades.append("TSLA")
# trades = ["AAPL", "TSLA"]
```

- list.insert(index, value) # inserts value at position; everything after shifts right
  - First arg = where to insert; second arg = what to insert
  - list.insert(0, x) — insert at the very beginning
  - Modifies in place, returns None

```
nums = [10, 30, 40]
nums.insert(1, 20)
# [10, 20, 30, 40]
```

- list.pop(index) # removes AND returns the item at index; pop() with no arg removes the last
  - Only method that both removes AND gives you back the item
  - list.pop() — no arg → removes and returns the last item
  - removed = list.pop(2) — save the removed item in a variable

```
items = ["a", "b", "c"]
last = items.pop()    # "c" removed, last="c"
mid  = items.pop(0)  # "a" removed, mid="a"
# items is now ["b"]
```

- list.remove(value) # removes the FIRST matching value; raises ValueError if not found
  - Removes by value, not by index
  - Only removes the FIRST occurrence — not all matches
  - Raises ValueError if the value is not in the list
  - Does NOT return the removed item (returns None)

```
fruits = ["apple", "banana", "cherry", "banana"]
fruits.remove("banana")
# ["apple", "cherry", "banana"]  — first one removed
```

- list.extend(other_list) # adds ALL items from another list to the end, one by one
  - Unlike append, unpacks the other list — items added individually, not nested
  - Modifies in place, returns None

⚠ append([3,4]) adds [3,4] as a single nested element; extend([3,4]) adds 3 then 4 separately

```
nums = [1, 2]
nums.extend([3, 4])    # [1, 2, 3, 4]

nums2 = [1, 2]
nums2.append([3, 4])   # [1, 2, [3, 4]]  <-- nested!
```

- list[:] # shallow copy of a list — changes to the copy do not affect the original
  - copy = original[:] — creates a separate list
  - Changing copy[0] = 99 does NOT change original
  - copy = original without [:] makes BOTH point to the same list

```
numbers = [1, 2, 3]
backup  = numbers[:]
numbers[0] = 99
# numbers = [99, 2, 3]
# backup  = [1, 2, 3]  (unchanged)
```

🔢 Sorting

- sorted(list) # returns a NEW sorted list; original list is untouched
  - sorted(scores) → new list, smallest to largest (default)
  - sorted(scores, reverse=True) → largest to smallest
  - sorted(list, key=func) — sort by a custom rule (pass a function)
    - Example: sort strings by length

```
def by_len(word):
    return len(word)

result = sorted(words, key=by_len)
# shortest to longest
```

  - Sort is stable: tied items keep their original relative order

```
scores = [400, 1200, 800, 600]
asc  = sorted(scores)
# [400, 600, 800, 1200]   scores unchanged
desc = sorted(scores, reverse=True)
# [1200, 800, 600, 400]   scores still unchanged
```

- list.sort() # sorts the list IN PLACE; returns None — the original list IS changed
  - list.sort() — ascending (smallest first)
  - list.sort(reverse=True) — descending (largest first)
  - list.sort(key=func) — custom sort key, same as sorted()

⚠ result = list.sort() → result is None! .sort() never returns the sorted list

```
nums = [3, 1, 4, 2]
nums.sort()
# nums is now [1, 2, 3, 4]

result = nums.sort()   # TRAP: result is None!
```

- .sort() vs sorted() # key difference: in-place mutation vs new list

  - .sort() → changes original, returns None
  - sorted() → original unchanged, returns new sorted list
  - Decision rule: "Do I want to change this list?" → use .sort()
  - Decision rule: "Do I want a sorted copy?" → use sorted()
  - Both are stable: tied items keep their original relative order

```
# .sort() — changes in place
nums = [3, 1, 2]
nums.sort()            # nums = [1, 2, 3]

# sorted() — new list, original untouched
nums = [3, 1, 2]
result = sorted(nums) # result=[1,2,3], nums=[3,1,2]
```